

Assignment 1: Wordle Solver

Course: Information Theory | Spring 2026

Overview

In this assignment you will build a **fast Wordle solver** based on **information theory**. Wordle is a word-guessing game where each guess produces colored feedback that reveals information about the hidden secret word. Your solver will compute which next guess provides the **maximum expected information** (measured by entropy / information gain), and recommend the best word to play.

Wordle Rules (Game Description)

You can play Wordle online [here](#). The secret word has **length 5**. You have a set of allowed guesses, and a set of possible secret answers.

After each guess, the game returns per-letter feedback:

- **Green (g)**: correct letter in the correct position.
- **Yellow (y)**: correct letter but in the wrong position.
- **Gray (r)**: letter does not appear in the secret word.

Duplicate-letter rule (important)

Wordle handles repeated letters carefully. The correct rule is:

1. Mark all **greens** first (exact matches). These consume letters from the target.
2. For remaining positions, mark a letter as **yellow** only if the target still contains unused occurrences of that letter.
3. Otherwise the letter is **gray**.

Example: target = APPLE, guess = ALLEY

Correct feedback is: g y r r y (because there is only one L in the target; extra L becomes gray).

Provided Data Files

We provide two [JSON files](#) (lists of lowercase words). You should not modify the original files.

- `dictionary_5_letter.json`: all **accepted guess words**
- `targets_5_letter.json`: all **possible secret answers**

What You Must Build

1) Precomputation: pattern matrix M

To make the solver fast, you will precompute a matrix:

$$M[i, j] = \text{pattern_code}(\text{guess}_i, \text{target}_j)$$

where:

- guess_i is the i -th word in the dictionary list (allowed guesses), size G
- target_j is the j -th word in the targets list (possible answers), size A
- M has shape (G, A)

2) Feedback encoding as an integer in $[0, 242]$

For a 5-letter guess, each position has 3 possible outputs:

$$r \rightarrow 0, \quad y \rightarrow 1, \quad g \rightarrow 2$$

Thus there are $3^5 = 243$ possible patterns.

Encode the full feedback pattern as a base-3 integer:

$$\text{code} = \sum_{k=0}^4 d_k \cdot 3^k$$

where $d_k \in \{0, 1, 2\}$ is the digit for position k (left-to-right).

- all gray `rrrrr` $\Rightarrow$ `code` = 0
- all green `ggggg` $\Rightarrow$ `code` = $2(1 + 3 + 9 + 27 + 81) = 242$

Hint: implement conversion both ways:

- `pattern_str_to_code("ryggr")` $\rightarrow$ integer
- `pattern_code_to_str(code)` $\rightarrow$ "ryggr"

3) Entropy-based ranking of guesses

At any point in the game you will have a set of remaining possible answers $C \subseteq \{\text{targets}\}$.

Assume the secret word W is uniformly distributed over C :

$$P(W = w) = \frac{1}{|C|}$$

Initial uncertainty:

$$H(W) = \log_2 |C|$$

For a given guess g , Wordle feedback is a random variable:

$$Y = f(g, W)$$

Compute the distribution of feedback patterns:

$$P(Y = p) = \frac{\#\{w \in C : f(g, w) = p\}}{|C|}$$

Then the entropy of the feedback is:

$$H(Y) = - \sum_p P(Y = p) \log_2 P(Y = p)$$

Decision rule (main strategy):

Choose the guess g^* that maximizes $H(Y)$ (highest expected information):

$$g^* = \operatorname{argmax}_{g \in \text{dictionary}} H(Y)$$

Information gain reporting:

You must print:

- prior entropy $H(W) = \log_2 |C|$
- best-guess expected feedback entropy $H(Y)$
- expected posterior entropy $H(W|Y) = H(W) - H(Y)$ (valid here because feedback is deterministic given W)
- information gain $I(W; Y) = H(Y)$

Interactive Input Format (for Autograding)

Each round, the autograder (or user) will provide exactly two lines:

Input line 1: guess word

A lowercase 5-letter word from the dictionary list:

`raise`

Input line 2: feedback

A 5-character feedback pattern using `r`, `y`, `g`:

`ryrgr`

Valid characters: `r`, `y`, `g`. Length must be exactly 5.

Important: your program must accept exactly this feedback format. (You can also print a user-friendly prompt, but input parsing must be strict.)

Output Requirements (for Autograding)

Your solver output must include analysis, but it must end in a **parseable final recommendation**.

The **last line of your program output** for each round must be:

`BEST=<word>`

where `<word>` is exactly a 5-letter lowercase guess.

Example:

<code>BEST=raise</code>

This must be the final line.

Required Files and Submission Format

You must submit a ZIP file named:

`<ID>_wordl.zip`

The `<ID>` part must be your student ID (digits only). No spaces.

What the ZIP must contain

Your ZIP must contain:

- **One main Python file with a fixed name:** `main.py`
- Any additional helper files you wrote (e.g., `utils.py`)

How we will run your code (Automatic grading)

The autograder will do:

```
unzip <ID>_wordl.zip
python main.py
```

Therefore:

- Running `python main.py` must start the solver.
- Your program must read input from `stdin` and write output to `stdout`.
- Your program must not require command-line arguments.
- you should include any files needed to run your code, including any precomputed matrices and dictionary files.

Academic Integrity

You may discuss ideas, but all submitted code must be your own. Copying code from online solvers or other students is not allowed.